

CSE234 Week 10:

Transformation and Sorting Algorithms

This week, we'll discuss the STL algorithms for modifying and sorting containers.

Sort

The `std::sort()` function is passed the start and end iterator of the range of a container to sort. The third parameter is optional, and is a binary predicate that returns true if the first element being compared is "less than" the second. If no predicate is passed, it uses the less than implemented for that data type. If no less than operator exists, and no predicate is passed, then `std::sort()` cannot be used. To sort a range in reverse order, you'd simply pass a predicate that returns true if the first element is greater than the second element, "tricking" the function into sorting the elements backwards.

```
#include <algorithm>
#include <vector>

int main() {
    std::vector<int> values = { 94, 53, 24, 37, 57, 2, 83, 2 };
    std::vector<int> copy(values);

    // sort smallest to largest
    // values == {2, 2, 24, 37, 53, 57, 83, 94}
    std::sort(values.begin(), values.end());

    // sort largest to smallest
    // copy == {94, 83, 57, 53, 37, 24, 2, 2}
    std::sort(copy.begin(), copy.end(), [](int x, int y){return x > y;

    return 0;
}
```

Removal

`std::remove()` and `std::remove_if()` remove items from a collection. Both take the start and end iterators of the collection.

`std::remove()` expects a possible element of the collection to be removed. `std::remove_if()` expects a predicate that returns true if the element needs to be removed. An iterator is returned to the location of the new end of the collection. Despite the items being "removed", the memory still exists. Typically, the corresponding container's `erase()` method must be invoked to free the unused part of the container that was invalidated by `remove()`

```
#include <algorithm>
#include <vector>

int main() {
    std::vector<int> values = { 0, 1, 3, -6, 1, 0, -5, 6, -2, 0 };
    // remove 0s from values
    // after operation:
    // values == { 1, 3, -6, 1, -5, 6, -2, ?, ?, ? }
    std::remove(values.begin(), values.end(), 0);
    // remove negatives from values
    // values = { 1, 3, 1, 6, ?, ?, ?, ?, ?, ? }
    auto it = std::remove_if(values.begin(), values.end(),
        [](int x) {
            // predicate returns true if element is ignored
            return x < 0;
        });
    // erase invalid items
    values.erase(it);
    return 0;
}
```

Move

`std::move()` moves elements from one collection to another. Its parameters are the same as those of `std::copy()`: the start and end iterator of the collection being copied and the start iterator of the destination (there is also a `std::move_backward()` that follows the same logic as `std::copy_backward()`). The result on the destination collection is the same. However, the source collection cannot be guaranteed to have the same elements as before the move. The source collection's memory is still valid and allocated, and it will have valid

elements, just not the same elements. There are some data types that cannot be copied, and therefore to put them in another collection they must be moved. An example would be `std::jthread` objects.

Note: There is another `std::move` defined in the "utility" header. This deals with C++ move semantics, which are not a topic of discussion in this class.

Swap

`std::swap()` takes two elements and swaps their values:

```
#include <algorithm>

int main() {
    int x = 5;
    int y = 3;
    std::swap(x, y);
    // x == 3
    // y == 5
    return 0;
}
```

`std::iter_swap()` swaps two elements of a collection using their iterators:

```
#include <algorithm>
#include <list>
#include <string>

int main() {
    std::list<std::string> todoList = {
        "mow grass",
        "water plants",
        "buy groceries",
        "pay bills"
    };
    // this swaps the order of "water plants" and "buy groceries"
    // in the list
    std::iter_swap(todoList.begin() + 1, todoList.begin() + 2);
    return 0;
}
```

There is also `std::swap_ranges()`, which takes the start and end iterator of a range of a collection, and the start iterator of the range it is

being swapped with.

```
#include <algorithm>
#include <string>

int main() {
    std::string str = "world hello";
    std::swap_ranges(str.begin(), str.begin() + 5, str.begin() + 6);
    // str == "hello world"
    return 0;
}
```

Replace

`std::replace()` operates similarly to `std::remove()`, but rather than deleting the element from the collection, it replaces it with another. Similarly, `std::replace_if()` replaces all elements that meet the provided criteria with another element.

```
#include <algorithm>
#include <vector>

int main() {
    std::vector<int> values = { 0, 4, 1, 6, 2, 5, 2, 5, 6 };
    // replaces all instances of 2 with -2
    // values == { 0, 4, 1, 6, -2, 5, -2, 5, 6 }
    std::replace(values.begin(), values.end(), 2, -2);
    // replaces all numbers greater than 3 with 3
    // values == { 0, 3, 1, 3, -2, 3, -2, 3, 3 }
    std::replace_if(values.begin(), values.end(),
        [](int x) { return x > 3; }, 3);

    return 0;
}
```

Unique

The `std::unique()` function will remove consecutive, duplicate elements from a range. The function takes iterators to the start and end of the range to remove duplicate elements from. Optionally, you can provide a predicate that returns true if the two elements are the same. Otherwise, the algorithm uses `operator==` to compare elements. An iterator to the new last element of the range is returned. Like `remove()`, a subsequent call to erase typically follows a call to

`unique()` to free the removed items.

```
#include <algorithm>
#include <vector>

int main() {
    std::vector<int> values = { 1, 1, 1, 2, 4, 2, 1, 2, 2, 3 };
    std::vector<int> uniqueValues(values);
    // uniqueValues == {1, 2, 4, 2, 1, 2, 3, x, x, x}
    std::unique(uniqueValues.begin(), uniqueValues.end());
}
```

Transform

`std::transform()` has two variants. The first takes the start and end iterator of a collection, an iterator for where to start storing the results of the transform, and a function callback to apply to each element of the collection. The callback should return the modified element. If you want to apply `std::transform()` to a whole collection (i.e., list or vector), simply pass the iterators returned by `begin()` and `end()` to the function. However, you can also pass other iterators if you only want to transform a subset of the collection (of course, the second iterator must come after the first). Below is an example that converts a single string to all uppercase.

```
#include <algorithm>
#include <string>

int main() {
    std::string str = "hello world";
    std::transform(str.begin(), str.end(), str.begin(),
        [](const char& c) {
            return toupper(c);
        });
    // str == "HELLO WORLD"
    return 0;
}
```

For the next version of `std::transform()`, I'll provide an example first and then explain:

```
#include <algorithm>
#include <cmath>
#include <vector>
```

```

// this will calculate the hypotenuses of triangles and store them in
// third vector using std::transform
int main() {
    // vector of the bases
    std::vector<int> bases = { 1, 2, 3, 4, 5, 6, 7, 8 };
    // vector of the heights
    std::vector<int> heights = { 2, 3, 4, 5, 12, 13, 24, 15 };
    // vector to store the hypotenuses in
    std::vector<double> hypotenuses(bases.size());
    // note that each parameter has a line comment above it
    // explaining its purpose
    std::transform(
        // the start iterator of the first range
        bases.begin(),
        // the end iterator of the first range
        bases.end(),
        // the start iterator of the second range
        // the second range is assumed to be at least
        // as large as the first
        heights.begin(),
        // the start iterator of the destination range
        hypotenuses.begin(),
        // the callback returns the data type of the destination
        // and takes a parameter from each of the input ranges
        // a will be an element of the first range,
        // b is from the second
        [](int a, int b) {
            //  $a^2 + b^2 = c^2 \Rightarrow c = \sqrt{a^2 + b^2}$ 
            // square root is the same as the 0.5 power
            // return value is stored in the destination range
            return pow(pow(a, 2) + pow(b, 2), 0.5);
        }
    );
    /*
        hypotenuses == {
                        2.23607,
                        3.60555,
                        5,
                        6.40312,
                        13,
                        14.3178,
                        25,
                        17
                    }
    */
    return 0;
}

```

This version of transform is basically used to combine elements of two

ranges. The iterators do not have to be from separate ranges, however:

```
#include <algorithm>
#include <vector>

int main() {
    std::vector<int> values = {1, 2, 3, 4, 5};
    std::vector<int> squares(values.size());

    // squares == {1, 4, 9, 16, 25}
    std::transform(values.begin(), values.end(),
                   values.begin(), values.end(),
                   squares.begin(), [](int x, int y) {
                       return x * y;
                   });

    // passing the same range as the output iterator
    // modifies the collection in place
    // values = {1, 4, 9, 16, 25}
    std::transform(values.begin(), values.end(),
                   values.begin(), values.end(),
                   values.begin(), [](int x, int y) {
                       return x * y;
                   });

    return 0;
}
```

Conclusion

This week, we discussed the STL algorithms for modifying and sorting containers.